

Cross-site Scripting

I. Giới thiệu chung

- Cross-Site Scripting (XSS) là một lỗ hổng bảo mật nghiêm trọng trên các ứng dụng web, cho phép kẻ tấn công chèn mã độc (thường là script) vào trang web, và mã này được thực thi phía client (trình duyệt người dùng).
- Mục tiêu chính của tấn công XSS là đánh cắp dữ liệu nhạy cảm như **cookies**, **session tokens**, và **mạo danh người dùng**. Khi cookie bị đánh cắp, kẻ tấn công có thể chiếm quyền truy cập vào tài khoản của người dùng mà không cần đăng nhập.
- XSS thường được thực hiện bằng các ngôn ngữ như JavaScript hoặc HTML.

II. Tấn công Cross-site Scripting (XSS)

- Cross-Site Scripting (XSS) là một lỗ hổng bảo mật web, trong đó hacker chèn mã độc hại (thường là JavaScript hoặc HTML) vào một trang web. Những mã độc này có thể phản chiếu trên trình duyệt của nạn nhân hoặc lưu trữ trong cơ sở dữ liệu, và được thực thi khi người dùng truy cập vào trang web.
- Nguyên nhân chính của XSS là do hệ thống không xác thực dữ liệu đầu vào đúng cách, cho phép mã độc thâm nhập vào dữ liệu đầu ra. Một số hình thức XSS phổ biến:
 - + Phía client: Tập lệnh độc hại thực thi trên trình duyệt người dùng.
 - + Trang web giả mạo: Lừa người dùng nhập thông tin nhạy cảm.
 - + Quảng cáo độc hại: Quảng cáo có chứa mã độc.
 - + Email độc hại: Lợi dụng lỗ hổng để tấn công.

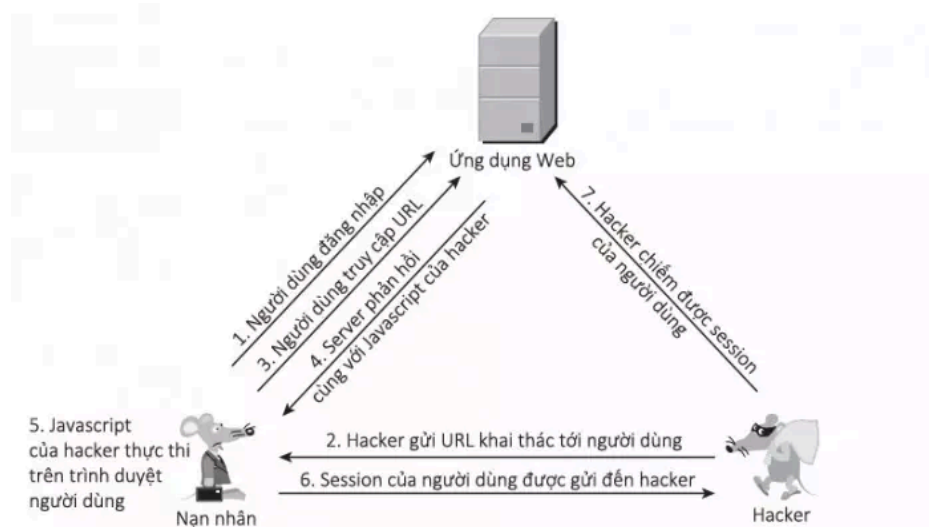
Ví dụ: Khi người dùng nhập mã độc vào ô tìm kiếm không an toàn, mã này sẽ được thực thi trên trình duyệt của họ.

III. Phân loại

Có 3 loại tấn công XSS chính :

a. **Reflected XSS** (tập lệnh độc hại xuất phát từ yêu cầu HTTP hiện tại)

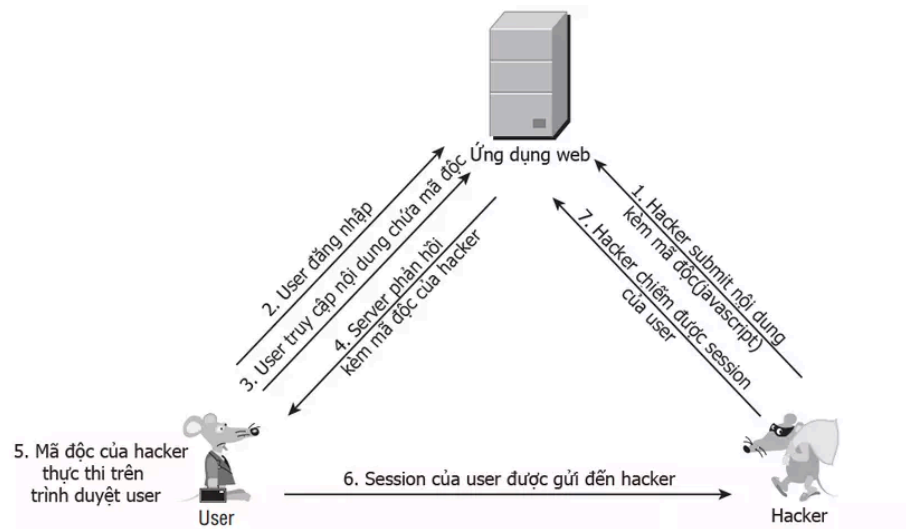
Một trong những cách được biết đến nhiều nhất là *chiếm phiên làm việc (session)* của người dùng, từ đó có thể truy cập được dữ liệu và chiếm được quyền của họ trên website



b. **Stored XSS** (mã độc hại xuất phát từ cơ sở dữ liệu của trang web)

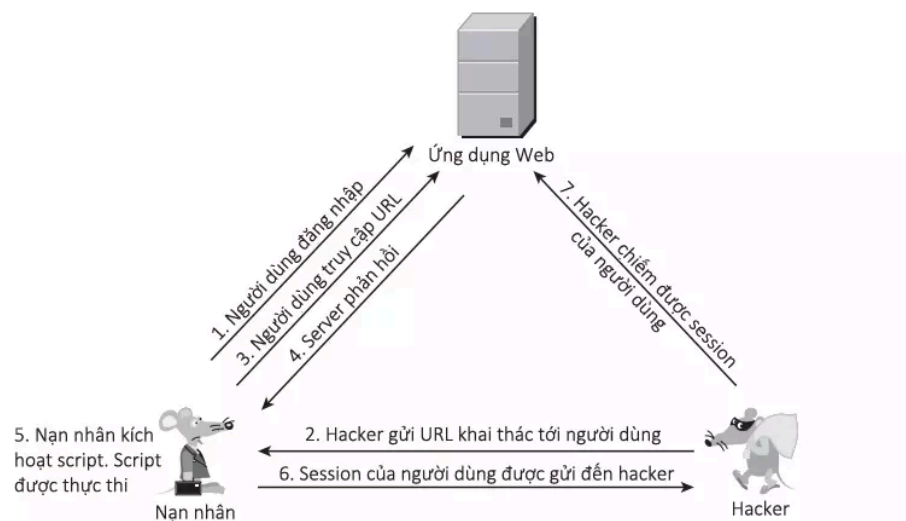
- + Lỗi này hướng đến nhiều nạn nhân hơn và xảy ra khi ứng dụng web không kiểm tra kỹ các dữ liệu đầu vào trước khi lưu vào cơ sở dữ liệu.
- + Người tấn công phải thực hiện tối thiểu 2 bước :

- B1 : Bên tấn công sẽ thông qua các điểm đầu vào (*form, input, textarea...*) không được kiểm tra kỹ để chèn vào CSDL các đoạn mã nguy hiểm.
- B2 : Người dùng truy cập vào ứng dụng web và thực hiện các thao tác liên quan đến dữ liệu được lưu này, đoạn mã của bên tấn công sẽ được thực thi trên trình duyệt người dùng.



c. **DOM-based XSS** (lỗ hổng tồn tại ở mã phía máy khách chứ không phải mã phía máy chủ)

Kỹ thuật này khai thác XSS dựa trên việc thay đổi cấu trúc DOM của tài liệu, cụ thể là HTML.



IV. Cách kiểm thử và ngăn chặn

- Để kiểm thử lỗ hổng XSS, phương pháp **kiểm thử hộp đen** có thể được áp dụng, tức là test mà không cần xem code. Các bước chính:



- + Xác định mục tiêu: Liệt kê các phần dễ bị XSS trên trang web và lập kế hoạch kiểm tra.
 - + Nhập kịch bản kiểm thử: Thử chèn mã độc như `<script>alert(document.cookie)</script>` vào các trường đầu vào.
 - + Phân tích kết quả: Nếu mã được thực thi, trang web có khả năng bị lỗ hổng XSS.
 - + Thử thêm các trường hợp đặc biệt: Kiểm thử cả các ký tự và dấu ngoặc đã được mã hóa.
- Tuy nhiên, việc **xem xét code** vẫn mang lại kết quả đáng tin cậy hơn :
- + Kiểm tra các đầu vào từ người dùng: Xem xét các điểm nhận dữ liệu từ người dùng như form, URL, hoặc API. Đảm bảo rằng dữ liệu này không được hiển thị trực tiếp trong HTML mà không qua kiểm tra và mã hóa.
 - + Kiểm tra các hàm xuất dữ liệu: Tìm các phần của code nơi dữ liệu đầu vào được hiển thị ra trình duyệt. Những hàm xuất dữ liệu như echo, print, hoặc các hàm tương tự trong các framework có thể dễ dẫn đến XSS nếu không qua mã hóa đúng cách.
 - + Mã hóa dữ liệu đúng cách: Đảm bảo dữ liệu đầu vào được mã hóa trước khi chèn vào HTML, JavaScript, hoặc các ngữ cảnh khác. Việc sử dụng các thư viện như OWASP ESAPI giúp mã hóa dễ dàng hơn.
 - + Kiểm tra các hệ thống bảo mật tích hợp: Xác nhận rằng framework hoặc thư viện đang sử dụng có tích hợp các biện pháp chống XSS như output encoding hoặc *context-sensitive sanitization*.
- Điều quan trọng là phân tích kỹ lưỡng kết quả để đảm bảo lỗ hổng được phát hiện và xử lý đúng cách.

V. Nguồn tham khảo

- <https://viblo.asia/p/ky-thuat-tan-cong-xss-va-cach-ngan-chan-YWOZr0Py5Q0>
- <https://portswigger.net/web-security/cross-site-scripting>



- https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Applications_Security_Testing/07-Input_Validation_Testing/01-Testing_for_Reflected_Cross_Site_Scripting

Mọi thắc mắc hay ý kiến đóng góp vui lòng liên hệ :

Email : r3dctf.info@gmail.com

@Design by R3D CTF Team